

Lab Session

Operator Overloading

Overview

C++ allows a programmer to redefine the functionality of built-in operators like +, -, and *, to operate on programmer-defined objects, a process known as **operator overloading**. Suppose a class TimeHrMn has data members hours and minutes. Overloading + would allow two TimeHrMn objects to be added with the + operator.

Example: Operator overloading allows use of operators like + on classes.

Without operator overloading	With operator overloading
<pre>TimeHrMn time1(3, 22); TimeHrMn time2(2, 50); TimeHrMn timeTot; timeTot.hours = time1.hours + time2.hours; timeTot.minutes = time1.minutes + time2.minutes; timeTot.Print();</pre>	<pre>TimeHrMn time1(3, 22); TimeHrMn time2(2, 50); TimeHrMn timeTot; timeTot = time1 + time2; timeTot.Print();</pre>
Console: H: 5, M: 72	Console: H: 5, M: 72

The sample result is achieved with simpler, more readable code.

Overloading TimeHrMn's + operator

To overload +, the programmer creates a member function named operator+. Although + requires left and right operands as in time1 + time2, the member function only requires the right operand (rhs: right-hand-side) as the parameter, because the left operand is the calling object. In other words, time1 + time2 is equivalent to the function call time1.operator+(time2), which is valid syntax but almost never used.

Example: TimeHrMn class implementation with overloaded + operator.

```
#include <iostream>
using namespace std;

class TimeHrMn {
public:
    TimeHrMn(int timeHours = 0, int timeMinutes = 0);
    void Print() const;
    TimeHrMn operator+(TimeHrMn rhs) ;
private:
    int hours;
    int minutes;
};

// Overload + operator for TimeHrMn
TimeHrMn TimeHrMn::operator+(TimeHrMn rhs) {
```

```
    TimeHrMn timeTotal;

    timeTotal.hours   = hours   + rhs.hours;
    timeTotal.minutes = minutes + rhs.minutes;

    return timeTotal;
}

TimeHrMn::TimeHrMn(int timeHours, int timeMinutes) {
    hours   = timeHours;
    minutes = timeMinutes;
}

void TimeHrMn::Print() const {
    cout << "H:" << hours << ", " << "M:" << minutes << endl;
}
```

Overloading the + operator multiple times

When an operator like + has been overloaded, the compiler determines which + operation to invoke based on the operand types. In `4 + 9`, the compiler sees two integer operands and thus applies the built-in + operation. In `time1 + time2`, where `time1` and `time2` are `TimeHrMn` objects, the compiler sees two `TimeHrMn` operands and thus invokes the programmer-defined function.

A programmer can define several functions that overload the same operator, as long as each involves different types so that the compiler can determine which to invoke. The code below overloads the + operator twice in the `TimeHrMn` class.

`main()` uses the + operator in 4 statements. The first + involves two `TimeHrMn` operands, so the compiler invokes the first `operator+` function ("A"). The second + involves `TimeHrMn` and `int` operands, so the compiler invokes the second `operator+` function ("B"). The third + involves two `int` operands, so the compiler invokes the built-in + operation. The fourth +, commented out, involves an `int` and `TimeHrMn` operands. Because no function has those operands ("B" has `TimeHrMn` and `int`, not `int` and `TimeHrMn`; order matters), that statement would generate a compiler error.

Example: Overloading the + operator multiple times.

```
#include <iostream>
using namespace std;

class TimeHrMn {
public:
    TimeHrMn(int timeHours = 0, int timeMinutes = 0);
    void Print() const;
    TimeHrMn operator+(TimeHrMn rhs);
    TimeHrMn operator+(int rhsHours);
private:
    int hours;
    int minutes;
};

// Operands: TimeHrMn, TimeHrMn. Call this "A"
```

```
TimeHrMn TimeHrMn::operator+(TimeHrMn rhs) {
    TimeHrMn timeTotal;

    timeTotal.hours = hours + rhs.hours;
    timeTotal.minutes = minutes + rhs.minutes;

    return timeTotal;
}

// Operands: TimeHrMn, int. Call this "B"
TimeHrMn TimeHrMn::operator+(int rhsHours) {
    TimeHrMn timeTotal;

    timeTotal.hours = hours + rhsHours;
    timeTotal.minutes = minutes; // Stays same

    return timeTotal;
}

TimeHrMn::TimeHrMn(int timeHours, int timeMinutes) {
    hours = timeHours;
    minutes = timeMinutes;

    return;
}

void TimeHrMn::Print() const {
    cout << "H:" << hours << ", " << "M:" << minutes << endl;
}

int main() {
    TimeHrMn time1(3, 22);
    TimeHrMn time2(2, 50);
    TimeHrMn sumTime;
    int num;

    num = 91;

    sumTime = time1 + time2; // Invokes "A"
    sumTime.Print();

    sumTime = time1 + 10;    // Invokes "B"
    sumTime.Print();

    cout << num + 8 << endl; // Invokes built-in add

    // timeTot = 10 + time1; // ERROR: No (int, TimeHrMn)

    return 0;
}
```

Problem 1: Type the output of operator overloading

```
#include <iostream>
using namespace std;
```

```
class InchSize {
public:
    InchSize(int wholeInches = 0, int sixteenths = 0);
    void Print() const;
    InchSize operator+(InchSize rhs);
private:
    int inches;
    int sixteenths;
};

InchSize InchSize::operator+(InchSize rhs) {
    InchSize totalSize;

    totalSize.inches = inches + rhs.inches;
    totalSize.sixteenths = sixteenths + rhs.sixteenths;

    // If sixteenths is greater than an inch, carry 1 to inches.
    if (totalSize.sixteenths >= 16) {
        totalSize.inches += 1;
        totalSize.sixteenths -= 16;
    }

    return totalSize;
}

InchSize::InchSize(int wholeInches, int sixteenthsOfInch) {
    inches = wholeInches;
    sixteenths = sixteenthsOfInch;
}

void InchSize::Print() const {
    cout << inches << " " << sixteenths << "/16 inches" << endl;
}

int main() {
    InchSize size1(5, 15);
    InchSize size2(9, 11);
    InchSize sumSize;

    sumSize = size1 + size2;

    sumSize.Print();

    return 0;
}
```

Problem 2: Operator overloading.

Four doubles are read from input, where the first two doubles are the liters and milliliters of volume1 and the second two doubles are the liters and milliliters of volume2. Complete the function to overload the + operator.

Ex: If the input is 11.0 276.0 2.0 194.5, then the output is:

11 liters, 276 milliliters 2 liters, 194.5 milliliters Sum: 13 liters, 470.5 milliliters

Note: The sum of two volumes is:

- the sum of the number of liters
- the sum of the number of milliliters

```
#include <iostream>
using namespace std;
```

```
class Volume {
public:
    Volume(double numLiters = 0.0, double numMilliliters = 0.0);
    void Print() const;
    Volume operator+(Volume rhs);
private:
    double liters;
    double milliliters;
};
```

```
Volume::Volume(double numLiters, double numMilliliters) {
    liters = numLiters;
    milliliters = numMilliliters;
}
```

```
// No need to accommodate for overflow or negative values
Volume Volume::operator+(Volume rhs) {
```

```
    /* Your code goes here */
```

```
}
```

```
void Volume::Print() const {
    cout << liters << " liters, " << milliliters << " milliliters";
}
```

```
int main() {
    double numLiters1;
    double numMilliliters1;
    double numLiters2;
    double numMilliliters2;
```

```
    cin >> numLiters1;
    cin >> numMilliliters1;
    cin >> numLiters2;
    cin >> numMilliliters2;

    Volume volume1(numLiters1, numMilliliters1);
    Volume volume2(numLiters2, numMilliliters2);

    Volume sum = volume1 + volume2;

    volume1.Print();
    cout << endl;
    volume2.Print();
    cout << endl;
    cout << "Sum: ";
    sum.Print();
    cout << endl;

    return 0;
}
```

Overloading the equality (==) operator

A programmer can overload the equality operator (==) to allow comparing objects of a programmer-defined class for equality. To overload ==, the programmer creates a function named operator== that returns bool and takes two const reference arguments of the class type for the left-hand-side and right-hand-side operands. Ex: To overload the == operator for a Review class, the programmer defines a function bool operator==(const Review& lhs, const Review& rhs).

The programmer must also determine when two objects are considered equal. In the Review class below, two Review objects are equal if the objects have the same rating and comment.

Example: Overloading the == operator.

```
bool operator==(const Review& lhs, const Review& rhs) {
    return (lhs.GetRating() == rhs.GetRating()) &&
           (lhs.GetComment() == rhs.GetComment());
}

// ...

if (myReview == bestReview) {
    cout << "Must be my favorite" << endl;
}
```

Problem 3: Type the output of the program using overloading operators.

```
#include <iostream>
```

```
#include <string>
using namespace std;

class Movie {
public:
    Movie(string movieTitle);
    void SetUpVotesAndDownVotes(int numUpVotes, int numDownVotes) {
        upVotes = numUpVotes;
        downVotes = numDownVotes;
    }
    string GetTitle() const { return title; }
    int GetUpVotes() const { return upVotes; }
    int GetDownVotes() const { return downVotes; }

private:
    string title;
    int upVotes;
    int downVotes;
};

Movie::Movie(string movieTitle) {
    title = movieTitle;
    upVotes = 0;
    downVotes = 0;
}

bool operator==(const Movie& movie1, const Movie& movie2) {
    return movie1.GetDownVotes() == movie2.GetDownVotes();
}

int main() {
    Movie movie1("It");
    Movie movie2("Taken");

    movie1.SetUpVotesAndDownVotes(8, 2);
    movie2.SetUpVotesAndDownVotes(8, 1);

    if (movie1 == movie2) {
        cout << "Equal" << endl;
    }
    else {
        cout << "Not equal" << endl;
    }

    return 0;
}
```